Database Course

Transactions & TCL

Lotfi NAJDI

Polydisciplinary Faculty of Taroudant

2025-2026

What is a Transaction?

A transaction is a **sequence of operations** performed as a single logical unit of work.
It's like a **"contract"** with your database 
Either ALL operations succeed, or ALL operations are rolled back

A Simple Analogy 🏦

Think of a transaction as a **bank transfer** between accounts.

- Step 1: Subtract money from Account A
- Step 2: Add money to Account B
- If Step 2 fails → Step 1 must be undone
- Money can’t just disappear!

Both steps succeed together, or both fail together

The ACID Properties 🧪

Every transaction must follow **ACID** principles:

Atomicity - All or nothing

Consistency - Data integrity is maintained

Isolation - Transactions don’t interfere with each other

Durability - Changes are permanent once committed

Transaction Control Language (TCL)

The Four Essential Commands

- **BEGIN** - Start a transaction
- **COMMIT** - Save all changes permanently
- **ROLLBACK** - Undo all changes
- **SAVEPOINT** - Create a checkpoint within a transaction

Basic Transaction Syntax

Starting and committing a transaction

```sql
BEGIN;
-- Your SQL operations here
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Rolling back a transaction

```sql
BEGIN;
-- Your SQL operations here
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
-- Oops, something went wrong!
ROLLBACK;
```

Real-World Example: Bank Transfer

Safe money transfer between accounts

```sql
BEGIN;

-- Check if source account has sufficient funds
SELECT balance FROM accounts WHERE id = 1;

-- Perform the transfer
UPDATE accounts
SET balance = balance - 100
WHERE id = 1 AND balance >= 100;

-- Check if the update affected exactly 1 row
-- If not, we'll rollback

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

COMMIT;
```

what are Savepoints?

- Named points within a transaction
- Allow partial rollbacks
- Useful for complex transactions

Savepoints: Partial Rollbacks

```sql
BEGIN;

-- First operation
INSERT INTO orders (customer_id, total) VALUES (123, 500);
SAVEPOINT after_order;

-- Second operation (might fail)
UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 456;
SAVEPOINT after_inventory;

-- Third operation
INSERT INTO shipments (order_id, address) VALUES (1001, '123 Main St');

-- If shipment fails, rollback to after inventory update
-- ROLLBACK TO after_inventory;
```

Savepoints: Partial Rollbacks

If you roll back to after_inventory

```
1 ROLLBACK TO after_inventory;
```

- Data kept: the orders insert and the inventory update remain.
- Data undone: only the shipments insert is undone.
- Savepoints: any savepoints created after after_inventory are discarded (there are none in your snippet). The after_inventory savepoint itself is still usable; after_order (earlier) is also still usable.
- In Postgres this is explicit: ROLLBACK TO SAVEPOINT destroys all savepoints created after the named one, but the named savepoint remains valid.

Savepoints: Partial Rollbacks

If you roll back to after_order

```
1  ROLLBACK TO after_order;
```

- Data kept: only the orders insert remains.
- Data undone: the inventory update and the shipments insert are undone.
- Savepoints: after_inventory (which was created after after_order) is discarded; after_order remains usable.

Clean-up and commit tips

- After you’re done, COMMIT; will persist whatever remains; ROLLBACK; (without a savepoint name) cancels the entire transaction and drops all savepoints.
- If you no longer need a savepoint (e.g., after_order), you can drop it without affecting data using RELEASE SAVEPOINT after_order;

Read Consistency

By default, each transaction sees a consistent snapshot of the database as of the start of the transaction. This means:

- Changes made by other transactions after the current transaction started are not visible.
- This ensures that reads within a transaction are stable and repeatable, preventing issues like non-repeatable reads and phantom reads.

Auto-commit vs Explicit Transactions

Auto-commit (default behavior)

```
1  -- Each statement is automatically committed
2  UPDATE customers SET email = 'new@email.com' WHERE id = 1;
3  -- Already committed!
```

Explicit transactions

```
1  -- You control when to commit
2  BEGIN;
3  UPDATE customers SET email = 'new@email.com' WHERE id = 1;
4  UPDATE customers SET phone = '555-1234' WHERE id = 1;
5  -- Both updates happen together or not at all
6  COMMIT;
```

Speaker notes